

DogLog

모바일 앱 개발 프로젝트 최종보고서

반려견 임시보호 일지 공유 및 입양 문의 관리 앱

항목	내용
앱명	DogLog
개발 형태	Android Java 앱 + Spring Boot 백엔드 + MySQL DBMS
제출 산출물	최종보고서 DOCX/PDF, build 폴더 제외 프로젝트 소스 ZIP
제외 항목	시연영상은 별도 촬영 완료로 본 산출물 생성 대상에서 제외

본 보고서는 기말 프로젝트 최종산출물 기준에 맞추어 주제, 완성 범위, 계획 대비 변경사항, 클래스 구성, 주요 구현 로직, DB 설계, AI 활용 및 디버깅 과정을 정리하였다.

1. 프로젝트 개요

DogLog는 임시보호자가 보호 중인 강아지를 등록하고 일지를 작성하면, 입양희망자가 강아지 정보와 일지를 확인한 뒤 문의 메시지를 보낼 수 있는 모바일 앱이다. 단순 게시판이 아니라 보호자, 강아지, 일지, 문의가 DB로 연결되는 구조를 목표로 하였고, Android 앱은 Spring Boot 서버를 통해 MySQL 데이터를 공유한다.

구분	구현 내용
주요 사용자	임시보호자(foster), 입양희망자(adopter)
핵심 흐름	회원가입/로그인 → 홈 목록 조회 → 강아지 상세 확인 → 일지 작성/조회 → 문의/답장
서버 통신	Android Volley로 10.0.2.2:8080의 Spring Boot API 호출
DB 공유	APP_USER, USER, DOG, DIARY, CONTACT_MESSAGE 테이블에 데이터 저장

주요 기능 요약

- 임시보호자는 본인 계정으로 등록한 강아지만 홈에서 보고, 해당 강아지에 대해서만 일지를 작성할 수 있다.
- 입양희망자는 모든 강아지를 조회할 수 있고, 일지 작성자 아이디를 확인한 뒤 문의 메시지를 보낼 수 있다.
- 강아지 신규 등록 시 홈 화면과 상세 화면에 표시될 사진을 함께 저장할 수 있다.
- 일지 작성 시 한국어 내용, 사진, 사료량, 배변량을 DB에 저장하고 상세 화면에서 조회한다.
- 마이페이지에서 로그인 상태 확인, 로그아웃, 받은 문의/보낸 문의 조회, 임시보호자의 답장 처리를 제공한다.
- 검색어, 품종 필터, 나이 필터를 적용하여 홈 목록을 빠르게 좁힐 수 있다.

2. 완성 및 미완성 범위

범위	상태	설명
회원가입/로그인	완성	회원 정보가 MySQL APP_USER 및 USER 테이블에 저장되고 DB 정보로 로그인 가능하다.
강아지 목록/검색/필터	완성	사용자 유형에 따라 목록 노출 범위를 다르게 처리하고 검색/품종/나이 필터를 적용한다.
강아지 등록/수정/삭제	완성	임시보호자 본인이 등록한 강아지에 한해 등록, 사진 저장, 상세 수정, 삭제가 가능하다.
일지 작성/조회/삭제	완성	본인 강아지에 대해서만 일지를 작성하며 작성자 아이디, 사진, 한국어 내용을 DB에 저장한다.
문의/답장	완성	입양희망자가 일지 작성자에게 문의하고 임시보호자가 마이페이지에서 답장할 수 있다.
시연영상	제외	사용자가 이미 촬영 완료한 항목이므로 본 자동 산출물 생성 범위에서 제외하였다.
운영 배포/보안 고도화	미구현	수업 프로젝트 범위 밖으로 서버 외부 배포, HTTPS, 비밀번호 해시화는 향후 개선 대상으로 남겼다.

3. 계획 대비 수정 사항

영역	초기 방향	최종 구현 방향
서버	JSP 파일을 Tomcat에서 직접 실행하는 방식	JSP 경로는 유지하되 Spring Boot 컨트롤러가 /ServerProject/*.jsp 요청을 처리하도록 변경했다.
DB	일부 화면 중심 저장	MySQL doglog DB를 사용하고 schema.sql로 USER, DOG, DIARY, CONTACT_MESSAGE 테이블을 자동 생성한다.
권한	모든 사용자가 동일한 목록 조회	임시보호자는 본인 강아지만, 입양희망자는 모든 강아지를 조회하도록 분리했다.
일지	상세 화면 중심 작성 흐름	홈 화면 우측 하단 작성 버튼에서 일지 작성 페이지로 이동하도록 UX를 수정했다.
사진	텍스트 정보 중심	사진 선택기를 추가하고 Base64 문자열을 DB에 저장하여 강아지/일지 사진을 표시한다.
문의	연락 기능 없음	일지 작성자 아이디 표시, 문의 메시지 저장, 마이페이지 답장 기능을 추가했다.

개발 환경

분류	사용 기술
Android	Java, AndroidX AppCompat, Material Components, ConstraintLayout, RecyclerView, Volley
Backend	Java 21, Spring Boot 3.3.5, Spring Web, Spring JDBC
Database	MySQL 8.x, utf8mb4 문자셋, H2 기반 백엔드 테스트
Build/Test	Gradle 8.x, JUnit 5, Spring Boot Test, MockMvc
Tool	Android Studio, Docker MySQL, ChatGPT/Codex 기반 디버깅 보조

4. 구현 클래스 목록 및 역할

Android 앱 주요 클래스

클래스	역할
MainActivity	하단 내비게이션을 구성하고 홈/마이페이지 Fragment 전환을 담당한다.
HomeActivity	강아지 목록을 서버에서 가져오고 로그인 상태별 버튼/목록 노출, 검색/필터링, 작성 페이지 이동을 처리한다.
DogAdapter	RecyclerView에 강아지 카드, 사진, 기본 정보를 표시하고 상세 페이지 Intent를 전달한다.
DogDetailActivity	강아지 상세 정보와 일지 목록을 표시하며 작성자 권한에 따른 수정/삭제와 사진 변경을 처리한다.
WriteActivity	신규 강아지 등록 또는 기존 강아지 선택 후 일지 작성, 사진 첨부, 사료량/배변량 수치 표시를 처리한다.
LoginActivity / SigninActivity	로그인과 회원가입 요청을 서버로 보내고 SharedPreferences에 사용자 상태를 저장한다.
MyPageFragment	로그인 사용자 정보, 로그아웃, 받은/보낸 문의 목록, 임시보호자 답장 기능을 제공한다.
MessageActivity	입양희망자가 일지 작성자에게 문의 메시지를 작성하여 서버에 저장한다.
DiaryAdapter / ContactMessageAdapter	일지 상세 다이얼로그와 문의 카드 UI, 삭제/연락/답장 액션을 연결한다.

클래스	역할
Dog, DiaryEntry, ContactMessage	서버 응답을 화면에 전달하기 위한 모델 클래스이다.
ServerConfig	에뮬레이터에서 호스트 서버로 접근하기 위한 10.0.2.2 기반 API 주소를 중앙 관리한다.

Backend 주요 클래스

클래스	역할
DogLogBackendApplication	Spring Boot 애플리케이션 시작점이다.
DogLogController	JSP 호환 URL 전체를 REST 엔드포인트처럼 받아 회원/강아지/일지/문의 CRUD를 처리한다.
Dog, Diary, ContactMessage, LoginResponse	컨트롤러가 JSON으로 반환하는 응답 DTO 역할을 수행한다.
DogLogControllerTest	회원가입/로그인, 한글 일지, 권한 제한, 문의/답장 흐름을 MockMvc로 검증한다.
LargePhotoUploadTest	큰 사진 Base64 데이터가 기본 Tomcat form 제한을 넘지 않고 저장되는지 검증한다.

5. DB 설계 및 테이블 설명

DBMS는 MySQL을 사용하며 Spring Boot 시작 시 schema.sql을 통해 필요한 테이블을 생성한다. 문자셋은 한국어 저장을 위해 utf8mb4와 utf8mb4_unicode_ci를 적용하였다.

테이블	주요 컬럼	용도
APP_USER	userId, userPassword, userEmail, userGender, userType	신규 회원가입 정보를 저장하는 주 사용자 테이블이다.
USER	userId, userPassword, userEmail, userGender, userType	기존 JSP/레거시 로그인 호환을 위해 유지한 사용자 테이블이다.
DOG	id, name, breed, age, status, fosterId, photoData	강아지 기본 정보, 등록된 임시보호자, 홈/상세 사진을 저장한다.
DIARY	id, fosterId, dogName, dateText, foodAmount, poopCount, content, photoData, createdAt	강아지별 일지, 작성자, 사료량/배변량, 한국어 본문, 사진을 저장한다.
CONTACT_MESSAGE	id, senderId, receiverId, dogName, title, content, replyContent, repliedAt, createdAt	입양희망자 문의와 임시보호자 답장을 저장한다.

테이블 관계 요약

- APP_USER.userId 또는 USER.userId는 DOG.fosterId, DIARY.fosterId, CONTACT_MESSAGE.senderId/receiverId와 논리적으로 연결된다.
- DOG.name과 DOG.fosterId 조합으로 특정 임시보호자의 강아지를 식별하고, DIARY는 dogName/fosterId를 함께 저장해 작성 권한을 확인한다.
- CONTACT_MESSAGE는 senderId와 receiverId를 분리하여 입양희망자와 임시보호자의 마이페이지 조회 방향을 구분한다.

DB 레코드 확인 방법

확인 대상	예시 SQL 또는 API	확인 의미
강아지 목록	SELECT name, breed, age, fosterId FROM DOG;	등록된 강아지와 임시보호자 소유 관계를 확인한다.
일지 목록	SELECT dogName, fosterId, foodAmount, poopCount, content FROM DIARY;	일지 작성자, 수치, 한국어 본문 저장 여부를 확인한다.
문의 메시지	SELECT senderId, receiverId, dogName, replyContent FROM CONTACT_MESSAGE;	문의와 답장 데이터가 같은 레코드에 연결되는지 확인한다.
앱 API	GET /ServerProject/GetDogList.jsp	앱 홈 화면에 표시되는 JSON 목록을 확인한다.

6. 주요 코드 블록 및 구현 로직

6.1 에뮬레이터 서버 주소 중앙 관리

```
public final class ServerConfig {  
    public static final String BASE_URL = "http://10.0.2.2:8080/ServerProject/";  
    public static String endpoint(String fileName) { return BASE_URL + fileName; }  
}
```

Android 에뮬레이터에서 PC의 localhost는 10.0.2.2로 접근해야 하므로 서버 주소를 ServerConfig에서 한 번만 관리한다.

6.2 사용자 유형별 홈 목록 분기

```
if ("foster".equals(userType) && !userID.isEmpty()) {  
    return url + "?fosterId=" + URLEncoder.encode(userID, "UTF-8");  
}  
return url;
```

임시보호자는 fosterId 파라미터로 본인 강아지만 조회하고, 입양희망자는 파라미터 없이 전체 강아지를 조회한다.

6.3 본인 강아지에 대해서만 일지 저장

```
if (!dogBelongsToFoster(dogName, ownerId)) { return "fail"; }  
jdbcTemplate.update("INSERT INTO DIARY (...) VALUES (?, ?, ?, ?, ?, ?, ?)",  
    ownerId, dogName, dateText, foodAmount, poopCount, content, photoData);
```

서버에서 dogName과 fosterId 소유 관계를 한 번 더 검증한 뒤 일지를 저장하여 API 직접 호출도 방어한다.

6.4 사진 선택 후 Base64 저장

```
photoPicker = registerForActivityResult(new ActivityResultContracts.GetContent(), uri -> {  
    selectedPhotoUri = uri;  
    imagePreview.setImageURI(uri);  
});  
String photoData = Base64.encodeToString(bytes, Base64.NO_WRAP);
```

에뮬레이터 갤러리 이미지를 선택해 미리보기로 표시하고, DB 공유를 위해 Base64 문자열로 변환한다.

6.5 문의 메시지와 답장 저장

```
INSERT INTO CONTACT_MESSAGE (senderId, receiverId, dogName, title, content)  
UPDATE CONTACT_MESSAGE SET replyContent = ?, repliedAt = CURRENT_TIMESTAMP  
WHERE id = ? AND receiverId = ?
```

문의와 답장을 같은 메시지 레코드에 연결하여 마이페이지에서 받은 문의와 보낸 문의를 모두 확인할 수 있다.

6.6 한글 데이터 저장을 위한 UTF-8 처리

```
application/x-www-form-urlencoded; charset=UTF-8  
server.servlet.encoding.charset=UTF-8  
useUnicode=true&characterEncoding=utf8
```

클라이언트 POST 본문, 서버 응답, MySQL 연결을 모두 UTF-8로 맞추어 한국어 일지를 안정적으로 저장한다.

7. GenAI, 오픈소스, 검색 활용 과정

본 프로젝트는 Android 공식 문서, Spring Boot/Gradle 오류 메시지, ChatGPT/Codex를 보조 도구로 활용하였다. 단순 복사보다 실제 프로젝트 변수명, DB 구조, 화면 흐름에 맞게 수정하고 테스트로 검증하는 데 초점을 두었다.

서버 통신 구조 전환

- 활용 리소스: ChatGPT/Codex, Spring Boot 공식 개념, Android 에뮬레이터 네트워크 검색
- 문제 상황: JSP/Tomcat을 매번 따로 실행해야 하고 에뮬레이터에서 localhost 통신이 실패했다.
- 초기 프롬프트/검색어: Android emulator localhost Spring Boot 10.0.2.2, JSP API를 Spring Boot에서 같은 경로로 받는 방법
- 해결 과정: /ServerProject/*.jsp 경로를 Spring Boot @GetMapping/@PostMapping으로 유지하고 Android는 10.0.2.2:8080으로 호출하도록 ServerConfig를 만들었다.
- 이해 및 성찰: 에뮬레이터의 localhost는 PC가 아니라 에뮬레이터 자신을 가리키므로 10.0.2.2를 사용해야 함을 이해했다.

한글 일지 저장 문제

- 활용 리소스: ChatGPT/Codex, MySQL utf8mb4 설정 검색
- 문제 상황: 일지 상세 내용에 한국어를 입력하면 저장/조회 과정에서 깨지거나 실패할 가능성이 있었다.
- 초기 프롬프트/검색어: Android Volley form POST Korean UTF-8, MySQL utf8mb4 Spring Boot characterEncoding
- 해결 과정: Volley 요청 Content-Type, Spring Boot encoding, MySQL JDBC URL, schema.sql 문자셋을 모두 UTF-8/utf8mb4로 맞추었다.
- 이해 및 성찰: 한글 문제는 클라이언트 인코딩, 서버 요청 해석, DB 문자셋이 모두 맞아야 해결된다.

사진 첨부 기능 구현

- 활용 리소스: Android ActivityResultContracts 문서, ChatGPT/Codex
- 문제 상황: 에뮬레이터 갤러리에서 사진을 선택하고 DB에 공유되게 저장해야 했다.
- 초기 프롬프트/검색어: Android Java image picker ActivityResultContracts Base64 upload Volley
- 해결 과정: GetContent 사진 선택기를 사용하고 InputStream을 byte 배열로 읽어 Base64 문자열로 변환했다. 서버는 photoData를 LONGTEXT로 저장한다.
- 이해 및 성찰: 수업 범위에서는 Base64 저장으로 기능을 완성할 수 있지만 실제 서비스에서는 파일 스토리지 분리가 더 적합하다.

권한 제한과 삭제 로직

- 활용 리소스: ChatGPT/Codex, 서버 권한 검증 사례 검색
- 문제 상황: 화면에서 버튼을 숨겨도 API를 직접 호출하면 다른 사용자의 데이터를 수정/삭제할 위험이 있었다.
- 초기 프롬프트/검색어: server side ownership check before delete, Spring JDBC delete with owner id
- 해결 과정: SaveDiary, DeleteDiary, DeleteDog에서 fosterId를 함께 받고 SQL WHERE 조건에 fosterId를 포함했다.
- 이해 및 성찰: 권한 처리는 UI 제어와 서버 검증을 함께 해야 안전하다.

8. 가장 어려웠던 버그 해결 및 디버깅 과정

8.1 앱 실행 시 서버 통신 실패

- 문제 상황: 앱은 실행되지만 서버 통신이 실패하고 홈 목록이 비어 있었다. Android 코드만 보면 URL 문자열은 정상처럼 보였기 때문에 서버, DB, 에뮬레이터 네트워크로 원인을 나누어 확인했다.
- 원인 파악: `http://localhost:8080/health`가 응답하지 않았고 8080 포트 LISTEN 상태도 없었다. 즉 앱 코드보다 백엔드 서버가 꺼져 있는 런타임 문제가 먼저였다.
- 해결 과정: Gradle `:backend:bootRun`으로 Spring Boot 서버를 실행하고 MySQL 컨테이너 상태를 확인했다. 이후 `/health`가 ok를 반환하고 `GetDogList.jsp`가 DB 강아지 JSON을 반환하는지 검증했다.
- 앱 검증: 앱을 재설치/실행한 뒤 logcat에서 `VolleyError`, 서버 통신 실패, `FATAL EXCEPTION`이 없는지 확인했다.
- 성찰: 모바일 앱 오류도 서버 프로세스, 포트, DB, 에뮬레이터 라우팅을 순서대로 분리해야 빠르게 원인을 찾을 수 있었다.

8.2 회원가입 데이터가 DB에 들어가지 않는 문제

- 문제 상황: 회원가입 후 로그인 화면에서는 성공처럼 보여도 실제 DB 회원 정보와 로그인 로직이 맞지 않았다.
- 원인 파악: 앱은 JSP 호환 테이블 `USER`를 기대하는 부분이 있었고 신규 백엔드 구조에서는 `APP_USER` 중심으로 저장되어 레거시 호환성이 떨어졌다.
- 해결 과정: 회원가입 시 `APP_USER`와 `USER` 양쪽에 저장하고, 로그인은 `APP_USER`를 우선 조회한 뒤 `USER` 테이블도 fallback으로 조회하도록 수정했다.
- 검증: `DogLogControllerTest`에서 `userCanRegisterAndLogin`, `userCanLoginWithExistingUserData` 테스트로 신규/기존 테이블 로그인을 모두 확인했다.

8.3 사진이 에뮬레이터에서 보이지 않는 문제

- 문제 상황: 사진을 선택해도 상세 화면이나 홈 화면에서 다시 표시되지 않았다.
- 원인 파악: 앱 내부 미리보기는 Uri로 가능하지만, 앱 재실행 또는 다른 화면에서는 같은 Uri 접근만으로는 DB 공유가 되지 않았다.
- 해결 과정: 선택 사진을 Base64 문자열로 변환해 `DOG.photoData` 또는 `DIARY.photoData`에 저장하고, 화면 표시 시 Base64를 Bitmap으로 디코딩하도록 통일했다.
- 검증: `LargePhotoUploadTest`로 큰 사진 데이터도 서버가 받을 수 있음을 확인했고, 홈/상세/일지 어댑터에서 `setPhoto` 로직으로 재표시되도록 했다.

9. 테스트 및 검증 결과

검증 항목	검증 방법	결과
백엔드 상태	GET /health	ok 응답 확인
강아지 목록 API	GET /ServerProject/GetDogList.jsp	DB 강아지 JSON 반환 확인
회원가입/로그인	MockMvc 테스트	APP_USER/USER 저장 및 로그인 성공
한글 일지	MockMvc UTF-8 테스트	한국어 본문 저장/조회 성공
권한 제한	MockMvc Save/Delete 테스트	작성자/소유자가 아니면 fail 반환

검증 항목	검증 방법	결과
문의/답장	MockMvc 메시지 테스트	문의 저장 및 답장 조회 성공
사진 업로드	LargePhotoUploadTest	3MB급 Base64 사진 폼 데이터 저장 성공
앱 런타임	에뮬레이터 실행 및 logcat 확인	VolleyError와 크래시 로그 없음

마무리

DogLog는 수업 요구 조건인 웹 서버와 DBMS 기반 CRUD를 만족하며, Android 앱 화면에서 생성한 회원/강아지/일지/문의 데이터가 MySQL DB에 공유되도록 구현되었다. 특히 임시보호자와 입양희망자 역할 분리, 작성자 권한 제한, 한국어 입력, 사진 첨부, 마이페이지 문의 답장까지 연결해 실제 사용 흐름을 갖춘 앱으로 완성하였다.